Secure Web Application Project

Amanda Lei and Christopher Chan

First Steps before running our application:

Local Setup & Execution

1) Generate Local SSL Certificates

This application requires HTTPS. Before running the app, you must generate a self-signed certificate. Run the following command in the root directory of the project: **openssl req -x509 -newkey rsa:4096 -nodes -out cert.pem -keyout key.pem -days 365**

(Note: Hit 'Enter' to skip through the prompts, but enter **localhost** for the prompt “Common Name”).

2) Start the Server: **python app.py** or **python3 app.py**

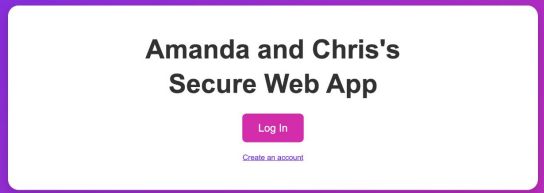
3) Access the App

Navigate to https://127.0.0.1:5000/ in your browser. You will likely see a "Your connection is not private" warning because the certificate is self-signed. Click "Advanced" and "Proceed to localhost" to view the app.

If get an address is already in use issue check device to clear port 5000 or change the port in the line at the end of app.py **app.run(ssl_context=('cert.pem', 'key.pem'), host='0.0.0.0', port=5000)**

Change 5000 to a different port number (ex. 5001)

Opening the Application



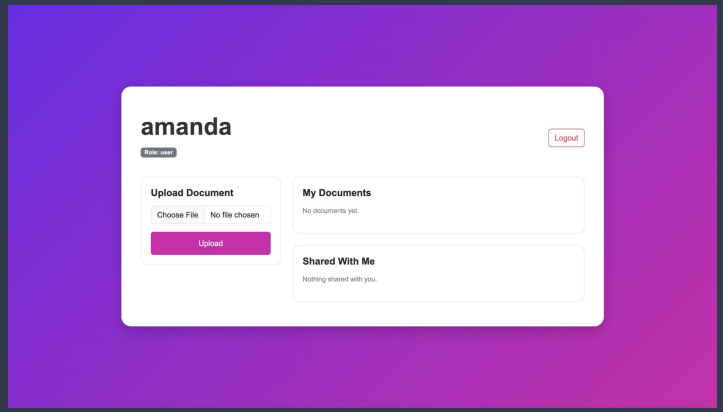
Logging In

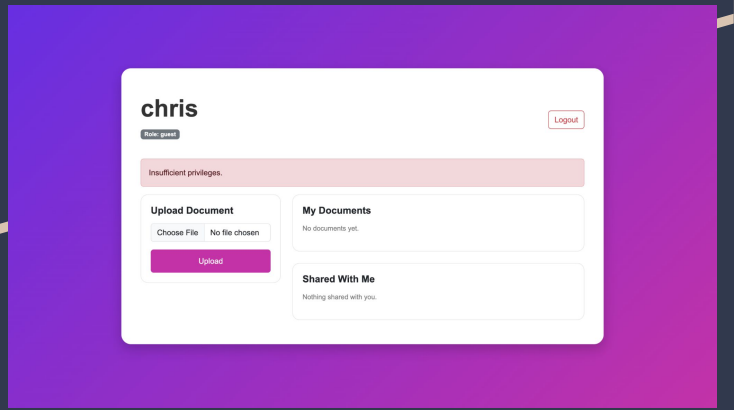
1) To login to our application, press the login button at the center of the screen
2) Insert your account name and password, and proceed to login

Creating an Account

1) To create an account, press the create an account button below the login button
2) To register, you will need to provide an email address, username and password
3) The username must fulfill the requirements:
    a) 3-20 characters
    b) Letters, numbers, and underscores only.
4) The email must fulfill the requirements:
    a) Must follow the pattern ___@___.___
5) The password must fulfill the requirements:
    a) 12 length minimum
    b) At least 1 uppercase letter
    c) At least 1 lowercase letter
    d) At least 1 number
    e) At least 1 special character (!@#$%^&*)
    f) Must match the password confirmation
6) Proceed to register your account

Using the dashboard



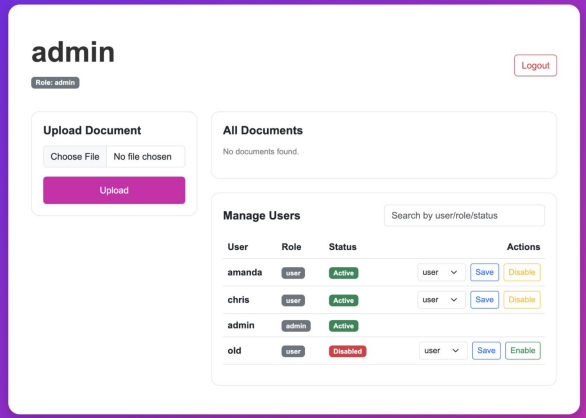


Every account starts as a guest and would need to be upgraded to a user in order to upload files and view shared files.

Upload a document as a user or admin

1) Press the upload file button and choose a file to upload
2) The file should be visible to other admins and users
3) The file should be able to share to other users or admins, and will not be able to share to guests (Note: It will show as user not found, if a guest or user not created)
4) In the second screenshot, there is a guest account that does not have the permissions to upload files, prompting an “insufficient privileges” message.

Admin View



1) To view the application as an admin, you will need to sign in using **u: admin** and **p: Password123!**
2) The admin dashboard will show and you can:
   a) Manage users permissions (ex. Change an account from user to guest/guest to admin)
   b) View all the documents uploaded
   c) Upload and share your own documents

Key Security Implementation (Access and Input)

Access Control:

Server-side validation ensures only the file owner, explicitly shared users, or admins can access a document.

Input Validation:

‘secure_filename()’ utilized to prevent Path Traversal attacks.

Strict file extension whitelisting (‘.txt’, ‘.pdf’, ‘.png’, ‘.jpg’).

Global ‘MAX_CONTENT_LENGTH’ (16MB) to prevent Denial of Service (DoS) via massive uploads.

Key Security Implementation (Configuration)

Encryption Implementation: Symmetric encryption utilizing the ‘cryptography.fernet’ library.

Comprehensive Security Headers:

- Content Security Policy (CSP)
- X-Frame-Options (Clickjacking defense)
- Strict-Transport-Security (HSTS)

Security Logging: Centralized, timestamped logging for all data access, authentication failures, and privilege warnings.

Key Security Implementations (Identity)

Authentication: Passwords hashed via ‘bcrypt’ (Cost Factor: 12).

Brute Force Protection: IP-based rate limiting (10 attempts/min) and automatic account lockouts after 5 failed login attempts.

Session Management:

- Cryptographically secure tokens generated via ‘secrets’.
- Strict 30-minute server-side idle timeout.
- Cookies flagged with ‘HttpOnly’, ‘Secure’, and ‘SameSite=Strict’.

Challenges and Lessons Learned

Challenge: Key Management: We accidentally tracked our master ‘secret.key’ in version control during early development.

Solution: Removed the file from Git tracking, added it to ‘.gitignore’, and successfully performed a full cryptographic key rotation.

Lesson: Client vs. Server Validation: We learned that HTML form validation (like password length) is only for User Experience.

Takeaway: Attackers bypass the browser. All security checks must be redundantly enforced on the Python backend.

Lesson: Session Trust: Relying on the browser to delete an expired cookie is a vulnerability; the server m